

Análisis de la Ionósfera con Apache Spark ML

Clasificación y Regresión sobre datos del radar Digisonda (IGP)

Práctica Calificada 04 · CC531 A – Análisis en Macrodatos · 2026-1

José Ismael Llanos Rosadio André Sánchez Vega Christian Luna Jaramillo

Universidad Nacional de Ingeniería
Facultad de Ciencias – Escuela de Ciencia de la Computación

Junio 2026

Índice de Contenidos

1 Introducción

2 Dataset

3 Marco Teórico

4 Metodología

5 Clasificación

6 Regresión

7 Gráficas de Pérdidas

8 Conclusiones

Introducción

Contexto

La **ionósfera** es la región atmosférica entre 60 y 1000 km de altitud donde la radiación solar ioniza gases, creando una concentración de electrones libres. Condiciona las comunicaciones HF, navegación GNSS/GPS y sistemas de radar.

El **IGP** monitorea la ionósfera peruana con un radar **Digisonda** en Jicamarca, publicando los datos en la PNDA. Esta práctica aplica **Apache Spark ML con Scala** para construir modelos de clasificación y regresión sobre esos datos.

Objetivos

- 1 Describir y caracterizar el dataset ionosférico (379,994 registros).
- 2 Construir un **Pipeline** de preprocesamiento reproducible.
- 3 Entrenar y comparar modelos de **clasificación** (4 modelos).
- 4 Entrenar y comparar modelos de **regresión** (4 modelos).
- 5 Optimizar hiperparámetros con **Grid-Search + CrossValidator**.

Dataset

Ionósfera – Radar Digisonda (IGP)
Fuente: PNDA · Jicamarca, Lima
Categoría: **Ciencia y Tecnología**

Dataset Seleccionado

Ficha del Dataset

Atributo	Detalle
Nombre	Ionósfera Digisonda – IGP
Entidad	Instituto Geofísico del Perú
Plataforma	PNDA
Ubicación	Jicamarca, Lurigancho, Lima
UBIGEO	150118
Período	2013–2026
Frecuencia	Cada 15 minutos
Formato	CSV (UTF-8)
Registros	379,994
Columnas	11

Parámetros físicos clave

- **foF2** (FREC_MAX_F2): frecuencia crítica de la capa F2 en MHz. Relacionada con la densidad máxima de electrones $N_{\max} \propto (\text{foF2})^2$.
- **h'F** (ALT_F): altura virtual de inicio de la capa F (km).
- **hmF2** (ALT_MAX_DENS): altura de máxima densidad electrónica (km).

Valor centinela

Las tres columnas físicas presentan valores nulos (NaN) cuando la Digisonda no pudo obtener un eco válido (de noche o perturbaciones ionosféricas). Se eliminan antes del modelado con `na.drop`.

URL de referencia

<https://www.datosabiertos.gob.pe/dataset/datos-de-la-ionosfera-obtenidos-con-el-radar-digisonda-instituto-geofisico-del-peru-igp>

Columnas del dataset (11 campos)

#	Campo	Tipo	Uso
0	FECHA_CORTE	INT	Constante
1	UBIGEO	INT	Constante
2	ANIO	INT	Feature
3	DEPARTAMENTO	STR	Constante
4	PROVINCIA	STR	Constante
5	DISTRITO	STR	Constante
6	FECHA.UTC	INT	Referencia
7	HORA.UTC	INT	Feature
8	FREC_MAX_F2	FLOAT	Target/Feature
9	ALT_F	FLOAT	Feature
10	ALT_MAX_DENS	FLOAT	Feature

Variables usadas en modelos

Features (entradas): HORA.UTC, ANIO, ALT_F, ALT_MAX_DENS

Target clasificación: NIVEL_ION (derivada de foF2)

Baja < 5.62 MHz

Normal [5.62 – 9.10] MHz

Alta > 9.10 MHz

Target regresión: FREC_MAX_F2 (valor continuo en MHz)

Nota sobre categorías

DEPARTAMENTO, PROVINCIA, DISTRITO y UBIGEO son constantes (único observatorio) $\Rightarrow$ varianza nula, no aportan información y se excluyen del modelo.

Estadísticas Descriptivas del Dataset

Estadísticas de columnas físicas

Campo	N válidos	Media	Mín	Máx
FREC_MAX_F2	298,736	7.41	1.90	15.99
ALT_F	298,736	254.77	66.0	720.26
ALT_MAX_DENS	284,099	319.42	129.6	719.62

Nulos y calidad

Campo	Nulos
FREC_MAX_F2	81,258 (21.4%)
ALT_F	81,258 (21.4%)
ALT_MAX_DENS	95,895 (25.2%)

Nulos = momentos sin eco válido de la Digisonda. Se eliminan con `na.drop` antes de modelar.

Discretización NIVEL_ION (clasificación)

Usando cuartiles Q1 y Q3 de `FREC_MAX_F2`:

Clase	Condición	Descripción
Baja	$< 5.62 \text{ MHz}$	Baja ionización
Normal	$[5.62, 9.10] \text{ MHz}$	Nivel típico
Alta	$> 9.10 \text{ MHz}$	Alta ionización

$Q1 = 5.62 \text{ MHz}$ · $Q3 = 9.10 \text{ MHz}$

Discretización por cuartiles $\Rightarrow$ clases balanceadas (25% / 50% / 25%).

Variación temporal

foF2 varía con la hora del día (ciclo diurno), la estación del año y el ciclo solar de 11 años $\Rightarrow$ relaciones **no lineales**.

Spark ML – Pipeline

Spark ML organiza el preprocesamiento y modelado en un **Pipeline** de etapas secuenciales:

- **StringIndexer**: convierte etiquetas texto en índices numéricos.
- **VectorAssembler**: agrupa columnas en un vector `features`.
- **MinMaxScaler** / **StandardScaler**: normaliza el vector de características.
- **Model**: clasificador o regresor.

Grid-Search + CrossValidator

ParamGridBuilder define la grilla de hiperparámetros.

CrossValidator evalúa cada combinación con k -fold cross-validation y selecciona el mejor modelo automáticamente.

Modelos de Clasificación

- **Árbol de Decisión**: particiones recursivas del espacio de características.
- **Random Forest**: conjunto de árboles, promedia predicciones.
- **Regresión Logística Multiclase**: modelo lineal probabilístico (softmax).
- **Naive Bayes**: modelo probabilístico con independencia condicional.

Modelos de Regresión

- **Regresión Lineal**: modelo lineal con regularización L2.
- **Random Forest Regressor**: árboles en ensemble, promedia valores.
- **GBT Regressor**: árboles boosteados con gradiente descendente.
- **XGBoost**: omitido por limitación de red NAT en WSL2.

Pipeline de Preprocesamiento

Pasos del Pipeline (clasificación)

- 1 **Carga:** `spark.read.csv` con `header=true`, `inferSchema=true`.
- 2 **Limpieza:** `na.drop` sobre `FREC_MAX_F2`, `ALT_F`, `ALT_MAX_DENS`.
- 3 **Target:** `withColumn` crea `NIVEL_ION` por cuartiles y elimina `FREC_MAX_F2` (evita fuga de datos).
- 4 **StringIndexer:** `NIVEL_ION` → `label` (numérico).
- 5 **VectorAssembler:** `[HORA_UTC, ANIO, ALT_F, ALT_MAX_DENS]` → `rawFeatures`.
- 6 **MinMaxScaler:** normaliza a `[0, 1]` → `features`.
- 7 **Split:** 80% train / 20% test (`seed = 1234`).

Diferencia Clasificación vs Regresión

Clasificación:

Scaler: `MinMaxScaler`

Split: 80/20

CV folds: 3

Regresión:

Target: renombrar `FREC_MAX_F2` → `label` (continuo)

Scaler: `StandardScaler` (`withStd=true`, `withMean=true`)

Split: 70/30

CV folds: 2

Naive Bayes:

Usa `rawFeatures` directamente (requiere valores no negativos).

Reproducibilidad

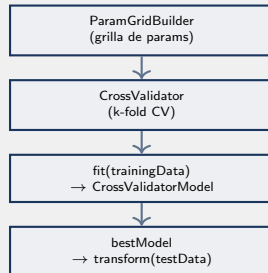
Semilla fija `seed = 1234` en `randomSplit` garantiza particiones idénticas en cada ejecución.

Grid-Search y Validación Cruzada

Grillas de hiperparámetros

Modelo	Hiperparámetro	Valores
Decision Tree	maxDepth	{5, 10}
Random Forest (clas.)	numTrees	{10, 20}
Reg. Logística	regParam	{0.1, 0.01}
Naive Bayes	smoothing	{1.0}
Linear Regression	regParam	{0.1}
Random Forest Reg.	numTrees	{10}
GBT Regressor	maxDepth	{3}

Flujo CrossValidator



Métrica de optimización

Clasificación: accuracy (MulticlassClassificationEvaluator)

Regresión: rmse (RegressionEvaluator)

Clasificación – Descripción y Pregunta

Tipo: Clasificación Supervisada Multiclase

¿Qué **nivel de ionización** (*Baja, Normal* o *Alta*) presenta la ionósfera, dados la hora UTC, el año y las alturas de la capa F?

Variables del modelo

Features (X): HORA.UTC, ANIO, ALT.F, ALT.MAX.DENS

Target (y): NIVEL.ION

- Baja: $\text{FREC_MAX_F2} < 5.62$
- Normal: $5.62 \leq \text{FREC_MAX_F2} \leq 9.10$
- Alta: $\text{FREC_MAX_F2} > 9.10$

Split de datos

80% entrenamiento / 20% prueba · seed = 1234

Cross-validation: **3 folds**

Modelos entrenados

- **Árbol de Decisión**
maxDepth $\in \{5, 10\}$
- **Random Forest**
numTrees $\in \{10, 20\}$
- **Regresión Logística Multiclase**
regParam $\in \{0.1, 0.01\}$, maxIter = 10
- **Naive Bayes**
smoothing = 1.0
Usa rawFeatures (no escala MinMax)

Métricas reportadas

Accuracy · Precision (weighted) · Recall (weighted) · F1-Score (weighted)

Clasificación – Código Pipeline I

clasificacion_completa.scala – Pipeline y Grid-Search

```
1 // 1. Carga y limpieza
2 val dfRaw = spark.read.option("header","true")
3   .option("inferSchema","true")
4   .csv("IGP_IonosferaRadarDigisonda_2013-2026_Dataset.csv")
5 val dfClean = dfRaw.na.drop(
6   Seq("FREC_MAX_F2","ALT_F","ALT_MAX_DENS"))
7
8 // 2. Crear variable objetivo NIVEL_ION (evita fuga de datos)
9 val dfClass = dfClean.withColumn("NIVEL_ION",
10   when(col("FREC_MAX_F2") < 5.62, "Baja")
11   .when(col("FREC_MAX_F2") > 9.10, "Alta")
12   .otherwise("Normal")).drop("FREC_MAX_F2")
13
14 // 3. Preparacion de features
15 val labelIndexer = new StringIndexer()
16   .setInputCol("NIVEL_ION").setOutputCol("label")
17 val assembler = new VectorAssembler()
18   .setInputCols(Array("HORA_UTC","ANIO","ALT_F","ALT_MAX_DENS"))
19   .setOutputCol("rawFeatures")
20 val scaler = new MinMaxScaler()
21   .setInputCol("rawFeatures").setOutputCol("features")
22
23 // 4. Division 80/20
24 val Array(trainingData, testData) =
25   dfClass.randomSplit(Array(0.8, 0.2), seed = 1234)
26
27 // 5. Modelos
28 val dt = new DecisionTreeClassifier()
29   .setLabelCol("label").setFeaturesCol("features")
30 val rf = new RandomForestClassifier()
```

Clasificación – Código Pipeline II

```
31 .setLabelCol("label").setFeaturesCol("features")
32 val lr = new LogisticRegression()
33 .setLabelCol("label").setFeaturesCol("features").setMaxIter(10)
34 val nb = new NaiveBayes()
35 .setLabelCol("label").setFeaturesCol("rawFeatures")
36
37 // 6. Pipelines
38 val pipeDT = new Pipeline()
39 .setStages(Array(labelIndexer, assembler, scaler, dt))
40 val pipeRF = new Pipeline()
41 .setStages(Array(labelIndexer, assembler, scaler, rf))
42 val pipeLR = new Pipeline()
43 .setStages(Array(labelIndexer, assembler, scaler, lr))
44 val pipeNB = new Pipeline()
45 .setStages(Array(labelIndexer, assembler, nb))
46
47 // 7. Grid-Search + CrossValidator (3 folds)
48 val evaluator = new MulticlassClassificationEvaluator()
49 .setLabelCol("label").setPredictionCol("prediction")
50
51 val gridDT = new ParamGridBuilder()
52 .addGrid(dt.maxDepth, Array(5, 10)).build()
53 val gridRF = new ParamGridBuilder()
54 .addGrid(rf.numTrees, Array(10, 20)).build()
55 val gridLR = new ParamGridBuilder()
56 .addGrid(lr.regParam, Array(0.1, 0.01)).build()
57 val gridNB = new ParamGridBuilder()
58 .addGrid(nb.smoothing, Array(1.0)).build()
59
60 val cvDT = new CrossValidator().setEstimator(pipeDT)
61 .setEvaluator(evaluator.setMetricName("accuracy"))
62 .setEstimatorParamMaps(gridDT).setNumFolds(3)
```

Clasificación – Código Pipeline III

```
63  
64 // 8. Entrenamiento y prediccion  
65 val modelDT = cvDT.fit(trainingData)  
66 val predDT = modelDT.transform(testData)
```

Clasificación – Resultados y Análisis

Tabla de métricas – Clasificación

Modelo	Acc.	Prec.	Rec.	F1
Árbol de Decisión	0.7685	0.7691	0.7685	0.7685
Random Forest	0.7418	0.7454	0.7418	0.7409
Reg. Logística	0.5729	0.5826	0.5729	0.5583
Naive Bayes	0.4386	0.4420	0.4386	0.3873

Interpretación

El **Árbol de Decisión** obtuvo el mejor desempeño (Accuracy = 0.7685). La relación entre alturas/hora y nivel de ionización es **no lineal**, favoreciendo modelos basados en árboles. La Regresión Logística (0.57) confirma la no-linealidad. Naive Bayes (0.44) falla porque ALT_F y ALT_MAX_DENS están correlacionadas, violando su supuesto de independencia.

[illegible]

Ejecución en spark-shell – Clasificación (Grid-Search, CrossValidator y métricas)

Regresión – Descripción y Pregunta

Tipo: Regresión Supervisada

¿Cuál es el valor numérico de la **frecuencia crítica foF2** (FREC_MAX_F2, en MHz) en función de la hora UTC, el año y las alturas de la capa F?

Variables del modelo

Features (X): HORA.UTC, ANIO, ALT_F, ALT_MAX_DENS

Target (y): FREC_MAX_F2 (continuo, MHz)
Rango: [1.90 – 15.99] MHz

Split de datos

70% entrenamiento / 30% prueba · seed = 1234
Cross-validation: **2 folds**
Scaler: **StandardScaler** (media 0, desv. estándar 1)

Modelos entrenados

- **Regresión Lineal**
regParam = 0.1 (L2)
- **Random Forest Regressor**
numTrees = 10
- **GBT Regressor**
maxDepth = 3, maxIter = 5
- **XGBoost Regressor**
Omitido: bloqueado por limitación NAT de WSL2 (RabitTracker). Código disponible en regresion_completa-c-XGB.scala

Métricas reportadas

RMSE · MSE · MAE · R^2

Regresión – Código Pipeline I

regresion_completa-s-XGBoostRegressor.scala – Pipeline

```
1 // 1. Carga y limpieza
2 val dfRaw = spark.read.option("header","true")
3   .option("inferSchema","true")
4   .csv("IGP_IonosferaRadarDigisonda_2013-2026_Dataset.csv")
5 val dfClean = dfRaw.na.drop(
6   Seq("FREC_MAX_F2","ALT_F","ALT_MAX_DENS"))
7
8 // 2. Target continuo (sin discretizacion)
9 val dfReg = dfClean.withColumnRenamed("FREC_MAX_F2","label")
10
11 // 3. Preparacion de features
12 val assembler = new VectorAssembler()
13   .setInputCols(Array("HORA_UTC","ANIO","ALT_F","ALT_MAX_DENS"))
14   .setOutputCol("rawFeatures")
15 val scaler = new StandardScaler()
16   .setInputCol("rawFeatures").setOutputCol("features")
17   .setWithStd(true).setWithMean(true)
18
19 // 4. Division 70/30
20 val Array(trainingData, testData) =
21   dfReg.randomSplit(Array(0.7, 0.3), seed = 1234)
22
23 // 5. Modelos nativos de Spark ML
24 val lr = new LinearRegression()
25   .setLabelCol("label").setFeaturesCol("features")
26 val rfr = new RandomForestRegressor()
27   .setLabelCol("label").setFeaturesCol("features")
28 val gbt = new GBTRRegressor()
29   .setLabelCol("label").setFeaturesCol("features").setMaxIter(5)
30
```

Regresión – Código Pipeline II

```
31 // 6. Pipelines
32 val pipeLR = new Pipeline().setStages(Array(assembler,scaler,lr))
33 val pipeRFR = new Pipeline().setStages(Array(assembler,scaler,rfr))
34 val pipeGBT = new Pipeline().setStages(Array(assembler,scaler,gbt))
35
36 // 7. Grid-Search + CrossValidator (2 folds)
37 val evaluatorReg = new RegressionEvaluator()
38   .setLabelCol("label").setPredictionCol("prediction")
39 val gridLR = new ParamGridBuilder()
40   .addGrid(lr.regParam, Array(0.1)).build()
41 val gridRFR = new ParamGridBuilder()
42   .addGrid(rfr.numTrees, Array(10)).build()
43 val gridGBT = new ParamGridBuilder()
44   .addGrid(gbt.maxDepth, Array(3)).build()
45
46 val cvRFR = new CrossValidator().setEstimator(pipeRFR)
47   .setEvaluator(evaluatorReg.setMetricName("rmse"))
48   .setEstimatorParamMaps(gridRFR).setNumFolds(2)
49
50 // 8. Entrenamiento
51 val modelRFR = cvRFR.fit(trainingData)
52 val predRFR = modelRFR.transform(testData)
```

Regresión – Resultados y Análisis

Tabla de métricas – Regresión

Modelo	RMSE	MSE	MAE	R ²
RF Regressor	1.3662	1.8666	1.0928	0.6859
GBT Regressor	1.4913	2.2239	1.1973	0.6257
Reg. Lineal	1.9191	3.6828	1.5481	0.3802
XGBoost	Omitido (WSL2 NAT)			

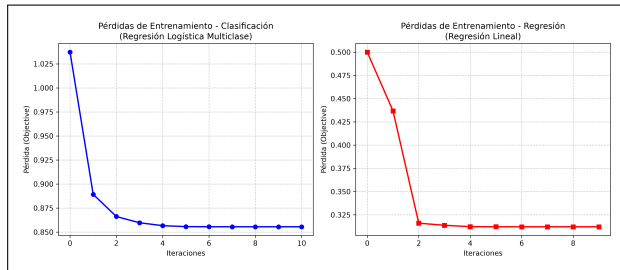
Interpretación

El **Random Forest Regressor** fue el mejor ($R^2 = 0.6859$, RMSE = 1.37 MHz), explicando el 68.6% de la varianza de foF2. El GBT lo sigue ($R^2 = 0.6257$). La Regresión Lineal solo explica el 38% porque foF2 varía **cíclicamente** (diurno + ciclo solar), relaciones no lineales que los modelos de conjunto capturan mejor. RMSE = 1.37 MHz frente al rango [1.9–16] MHz es razonable con solo 4 variables predictoras.

[illegible]

Ejecución en spark-shell – Regresión (Grid-Search, CrossValidator y métricas)

Gráficas de Pérdidas de Entrenamiento



Curvas de pérdida – Reg. Logística Multiclase (izq.) y Reg. Lineal (der.)

Pérdida Clasificación (Reg. Logística)

Optimizador: **L-BFGS**

Inicio: ≈ 1.037

Convergencia: ≈ 0.855 tras ~ 5 iteraciones
`objectiveHistory` ↓ suavemente

Pérdida Regresión (Reg. Lineal)

Inicio: ≈ 0.500

Convergencia: ≈ 0.312 tras ~ 4 iteraciones
Caída más pronunciada que clasificación

Interpretación

Ambas curvas descienden rápidamente y se estabilizan, evidenciando **correcta convergencia** del optimizador L-BFGS. No se observa sobreajuste ni divergencia, lo que valida el pipeline de preprocesamiento y la selección de hiperparámetros.

Entregables del Proyecto

Archivos de código y datos

Archivo	Descripción
clasificacion_completa.scala	Pipeline + Grid-Search de 4 clasificadores
regresion_completa-s-XGBoost.scala	Pipeline + Grid-Search de 3 regresores
regresion_completa-c-XGBoost.scala	Versión con XGBoost (try-catch WSL2)
summary.py	Análisis exploratorio con Pandas
graficar_perdidas.py	Generación de curvas de pérdida con matplotlib
graficos_perdidas_modelos.png	Gráfica de convergencia de optimizadores
IGP_Ionosfera...Dataset.csv	Dataset original (379,994 registros)
DiccionarioDatosDigisonda.xlsx	Diccionario de datos oficial IGP

Resumen de mejores modelos

Tarea	Mejor modelo	Métrica principal
Clasificación	Árbol de Decisión (maxDepth=10)	Accuracy = 0.7685
Regresión	Random Forest Regressor (numTrees=10)	$R^2 = 0.6859$

Hallazgos principales

- El **Árbol de Decisión** fue el mejor clasificador (Accuracy = 0.7685), superando a modelos lineales y probabilísticos.
- El **Random Forest Regressor** fue el mejor regresor ($R^2 = 0.6859$; RMSE = 1.37 MHz), explicando el 68.6% de la varianza de foF2.
- La naturaleza **no lineal** del fenómeno ionosférico (ciclo diurno + solar) favorece claramente a los modelos basados en árboles.
- El **balance de clases** por cuartiles (Q1/Q3) produjo clases bien distribuidas, lo que se refleja en la consistencia entre Precision, Recall y F1.

Aprendizajes técnicos

- **Spark ML Pipeline** permite encadenar preprocesamiento y modelado de forma reproducible y escalable.
- **Grid-Search + CrossValidator** automatiza la selección de hiperparámetros sin sesgo en los datos de prueba.
- **XGBoost** requiere conectividad de red (RabbitTracker) no disponible en WSL2 NAT; documentado con try-catch.
- Eliminar `FREC_MAX_F2` antes de clasificar es **crítico** para evitar fuga de datos (data leakage).